

FuFan-OpenClaw 开发需求文档 (PRD)

FuFan-OpenClaw 开发需求文档 (PRD)

一、项目介绍

- 功能与目标定位
- 项目核心技术架构

二、内置工具

- 命令行操作工具 (Command Line Interface)
- Python 代码解释器 (Python REPL)
- Fetch 网络信息获取
- 文件读取工具 (File Reader)
- RAG 检索工具 (Hybrid Retrieval)
- Browser Use 浏览器操作工具

三、fufan OpenClaw 的 Agent Skills 系统

- Agent Skills 基础功能介绍
- Agent Skills 载入与执行流程
 - Agent Skills 读取流程 (Bootstrap)
 - Agent Skills 调用流程 (Execution)

四、fufan OpenClaw 对话记忆管理系统设计

- 本地优先原则
- 系统提示词 (System Prompt) 构成
- AGENTS.md 的默认配置 (核心修正)
- 会话存储 (Sessions)

五、后端 API 接口规范 (FastAPI)

- 核心对话接口
- 文件管理接口 (用于前端编辑器)
- 会话管理接口
- Token 统计接口
- 对话压缩接口
- RAG 配置接口
- 技能管理接口

六、前端开发要求

- 设计理念与布局架构
- 技术栈
- UI/UX 风格规范
- 核心功能

七、项目目录结构参考

一、项目介绍

1. 功能与目标定位

FuFan-OpenClaw 是一个基于 **Python** 重构的、轻量级且高度透明的 AI Agent 系统，旨在复刻并优化 OpenClaw（原名 Moltbot/Clawdbot）的核心体验。

本项目不追求构建庞大的 SaaS 平台，而是致力于打造一个**运行在本地的、拥有"真实记忆"的数字副手**。其核心差异化定位在于：

- 文件即记忆 (File-first Memory)**：摒弃不透明的向量数据库，回归最原始、最通用的 Markdown/JSON 文件系统。用户的每一次对话、Agent 的每一次反思，都以人类可读的文件形式存在。

- **技能即插件 (Skills as Plugins)**: 遵循 Anthropic 的 Agent Skills 范式, 通过文件夹结构管理能力, 实现"拖入即用"的技能扩展。
- **透明可控**: 所有的 System Prompt 拼接逻辑、工具调用过程、记忆读写操作对开发者完全透明, 拒绝"黑盒"Agent。

2. 项目核心技术架构

本项目要求完全采用 **前后端分离** 架构, 后端作为纯 API 服务运行。

- **后端语言**: Python 3.10+ (强制使用 Type Hinting)。
- **Web 框架**: **FastAPI** (提供 RESTful 接口, 支持异步处理)。
- **Agent 编排引擎**: **LangChain 1.x (Stable Release)**.
 - **核心 API**: 必须使用 `create_agent` API (`from langchain.agents import create_agent`)。这是 LangChain 1.0 版本发布的最新标准 API, 用于构建基于 Graph 运行时的 Agent。
 - **核心说明**: 严禁使用旧版的 `AgentExecutor` 或早期的 `create_react_agent` (旧链式结构)。`create_agent` 底层虽然基于 LangGraph 运行时, 但提供了更简洁的标准化接口, 本项目应紧跟这一最新范式。
- **RAG 检索引擎**: **LlamaIndex (LlamaIndex Core)**.
 - 用于处理非结构化文档的混合检索 (Hybrid Search), 作为 Agent 的知识外挂。
- **模型接口**: 兼容 OpenAI API 格式 (支持 OpenRouter, DeepSeek, Claude 等模型直连)。
- **数据存储**: 本地文件系统 (Local File System) 为主, 不引入 MySQL/Redis 等重型依赖。

二、内置工具

FuFan-OpenClaw 在启动时, 除了加载用户自定义的 Skills 外, 必须内置以下 6 个核心基础工具 (Core Tools)。根据"优先使用 LangChain 原生工具"的原则, 技术选型更新如下:

1. 命令行操作工具 (Command Line Interface)

- **功能描述**: 允许 Agent 在受限的安全环境下执行 Shell 命令。
- **实现逻辑**:
 - **直接使用 LangChain 内置工具**: `langchain_community.tools.ShellTool`。
 - **配置要求**:
 - 初始化时需配置 `root_dir` 限制操作范围 (沙箱化), 防止 Agent 修改系统关键文件。
 - 需预置黑名单拦截高危指令 (如 `rm -rf /`)。
- **工具名称**: `terminal`。

2. Python 代码解释器 (Python REPL)

- **功能描述**: 赋予 Agent 逻辑计算、数据处理和脚本执行的能力。
- **实现逻辑**:
 - **直接使用 LangChain 内置工具**: `langchain_experimental.tools.PythonREPLTool`。
 - **配置要求**:
 - 该工具会自动创建一个临时的 Python 交互环境。

- **注意：**由于 `PythonREPLTool` 位于 `experimental` 包中，需确保依赖项安装正确。
- **工具名称：** `python_repl`。

3. Fetch 网络信息获取

- **功能描述：**用于获取指定 URL 的网页内容，Agent 联网的核心。
- **实现逻辑：**
 - **直接使用 LangChain 内置工具：** `langchain_community.tools.RequestsGetTool`。
 - **增强配置 (Wrapper)：**
 - 原生 `RequestsGetTool` 返回的是原始 HTML，Token 消耗巨大。
 - **必须封装：**建议继承该类或创建一个 Wrapper，在获取内容后使用 `BeautifulSoup` 或 `html2text` 库清洗数据，仅返回 Markdown 或纯文本内容。
- **工具名称：** `fetch_url`。

4. 文件读取工具 (File Reader)

- **功能描述：**用于精准读取本地指定文件的内容。这是 Agent Skills 机制的核心依赖，用于读取 `SKILL.md` 的详细说明。
- **实现逻辑：**
 - **直接使用 LangChain 内置工具：**
`langchain_community.tools.file_management.ReadFileTool`。
 - **配置要求：**
 - 必须设置 `root_dir` 为项目根目录，严禁 Agent 读取项目以外的系统文件。
- **工具名称：** `read_file`。

5. RAG 检索工具 (Hybrid Retrieval)

- **功能描述：**当用户询问具体的知识库内容（非对话历史）时，Agent 可调用此工具进行深度检索。
- **技术选型：** `LlamaIndex`。
- **实现逻辑：**
 - **索引构建：**支持扫描指定目录（如 `knowledge/`）下的 PDF/MD/TXT 文件，构建本地索引。
 - **混合检索：**必须实现 **Hybrid Search**（关键词检索 BM25 + 向量检索 Vector Search）。
 - **持久化：**索引文件需持久化存储在本地（`storage/`）。
- **工具名称：** `search_knowledge_base`。

6. Browser Use 浏览器操作工具

- **功能描述：**允许 Agent 通过 Playwright 驱动的浏览器执行复杂的网页交互任务（如表单填写、页面导航、信息提取等）。
- **实现逻辑：**
 - 基于 `browser-use` 库，封装为 LangChain BaseTool。
 - 接受自然语言任务描述，内部调用 Browser Use Agent 执行浏览器操作。
 - 返回操作结果文本。
- **依赖：** `browser-use`, `playwright`（安装后需执行 `playwright install chromium`）。

- 工具名称: `browser_use`。

三、fufan OpenClaw 的 Agent Skills 系统

1. Agent Skills 基础功能介绍

fufan OpenClaw 的 Agent Skills 遵循 "Instruction-following" (指令遵循) 范式, 而非传统的 "Function-calling" (函数调用) 范式。这意味着 Skills 本质上是教会 Agent 如何使用基础工具 (如 Python/Terminal) 去完成任务的说明书, 而不是预先写好的 Python 函数。

Agent Skills 以文件夹形式存在于 `backend/skills/` 目录下。

2. Agent Skills 载入与执行流程

2.1 Agent Skills 读取流程 (Bootstrap)

在 Agent 启动或会话开始时, 系统扫描 `skills` 文件夹, 读取每个 `SKILL.md` 的元数据 (Frontmatter), 并将其汇总生成 `SKILLS_SNAPSHOT.md`。

`SKILLS_SNAPSHOT.md` 示例:

```
<available_skills>
  <skill>
    <name>get_weather</name>
    <description>获取指定城市的实时天气信息</description>
    <location>./backend/skills/get_weather/SKILL.md</location>
  </skill>
</available_skills>
```

注意: `location` 使用相对路径。

2.2 Agent Skills 调用流程 (Execution)

这是本系统最独特的地方:

1. **感知**: Agent 在 System Prompt 中看到 `available_skills` 列表。
2. **决策**: 当用户请求"查询北京天气"时, Agent 发现 `get_weather` 技能匹配。
3. **行动 (Tool Call)**: Agent **不调用** `get_weather()` 函数 (因为它不存在), 而是调用 `read_file(path="./backend/skills/get_weather/SKILL.md")`。
4. **学习与执行**: Agent 读取 Markdown 内容, 理解操作步骤 (例如: "使用 `fetch_url` 访问某天气 API" 或 "使用 `python_repl` 运行以下代码"), 然后**动态调用** **Core Tools** (Terminal/Python) 来完成任务。

四、fufan OpenClaw 对话记忆管理系统设计

1. 本地优先原则

所有记忆文件 (Markdown/JSON) 均存储在本地文件系统, 确保完全的数据主权和可解释性。

2. 系统提示词 (System Prompt) 构成

System Prompt 由以下 6 部分动态拼接而成（按顺序）：

1. `SKILLS_SNAPSHOT.md` (能力列表)
2. `SOUL.md` (核心设定)
3. `IDENTITY.md` (自我认知)
4. `USER.md` (用户画像)
5. `AGENTS.md` (行为准则 & 记忆操作指南)
6. `MEMORY.md` (长期记忆)

截断策略：如果拼接后 Token 超出模型限制（或单文件超 20k 字符），需对超长部分进行截断并在末尾添加 `...[truncated]` 标识。

3. AGENTS.md 的默认配置 (核心修正)

由于 Agent 默认并不知道它是通过“阅读文件”来学习技能的，因此必须在初始化时生成一个包含明确指令的 `AGENTS.md`。

- **必须包含的元指令 (Meta-Instructions)：**

```
# 操作指南

## 技能调用协议 (SKILL PROTOCOL)
你拥有一个技能列表 (SKILLS_SNAPSHOT)，其中列出了你可以使用的能力及其定义文件的位置。
**当你使用某个技能时，必须严格遵守以下步骤：**
1. 你的第一步行动永远是使用 `read_file` 工具读取该技能对应的 `location` 路径下的 Markdown 文件。
2. 仔细阅读文件中的内容、步骤和示例。
3. 根据文件中的指示，结合你内置的 Core Tools (terminal, python_repl, fetch_url) 来执行具体任务。
**禁止**直接猜测技能的参数或用法，必须先读取文件！

## 记忆协议
...

## Canvas 输出协议
当用户请求创建可视化内容（网页、图表、仪表盘等），将完整 HTML 包裹在 `` 标签内输出。
HTML 必须是自包含的 (inline CSS/JS)，不要使用外部引用。
```

4. 会话存储 (Sessions)

- **路径：** `backend/sessions/{session_name}.json`
- **格式：** JSON 对象，包含 `title`, `created_at`, `updated_at`, `messages` 字段。`messages` 是一个数组，包含 `user`, `assistant`, `tool` (function calls) 类型的完整消息记录。
- **压缩：** 支持对话历史压缩，压缩后的摘要存储在 `compressed_context` 字段中，原始消息归档至 `sessions/archive/`。

五、后端 API 接口规范 (FastAPI)

后端服务作为独立进程运行，负责 Agent 逻辑、文件读写和状态管理。

- **服务端口:** 8002
- **基础 URL:** http://localhost:8002

1. 核心对话接口

- **Endpoint:** POST /api/chat
- **功能:** 发送用户消息，获取 Agent 回复。
- **Request:**

```
{
  "message": "查询一下北京的天气",
  "session_id": "main_session",
  "stream": true
}
```

- **Response:** 支持 SSE (Server-Sent Events) 流式输出，事件类型包括：
 - token — 流式文本输出
 - tool_start — 工具调用开始
 - tool_end — 工具调用结束
 - new_response — 新的响应段开始
 - retrieval — RAG 检索结果
 - canvas — Canvas HTML 内容
 - title — 自动生成的会话标题
 - done — 响应完成
 - error — 错误信息

2. 文件管理接口 (用于前端编辑器)

- **Endpoint:** GET /api/files
 - **Query:** path=memory/MEMORY.md
 - **功能:** 读取指定文件的内容。
- **Endpoint:** POST /api/files
 - **Body:** { "path": "...", "content": "..." }
 - **功能:** 保存对 Memory 或 Skill 文件的修改。

3. 会话管理接口

- GET /api/sessions — 获取所有历史会话列表
- POST /api/sessions — 创建新会话
- PUT /api/sessions/{id} — 重命名会话
- DELETE /api/sessions/{id} — 删除会话

- `GET /api/sessions/{id}/messages` — 获取会话原始消息 (含 system prompt)
- `GET /api/sessions/{id}/history` — 获取会话历史 (含 tool_calls)

4. Token 统计接口

- `GET /api/tokens/session/{id}` — 获取会话 token 统计
- `POST /api/tokens/files` — 获取指定文件列表的 token 统计

5. 对话压缩接口

- `POST /api/sessions/{id}/compress` — 压缩 50% 历史对话为摘要

6. RAG 配置接口

- `GET /api/config/rag-mode` — 获取 RAG 模式状态
- `PUT /api/config/rag-mode` — 开启/关闭 RAG 模式

7. 技能管理接口

- `GET /api/skills` — 获取技能列表
- `DELETE /api/skills/{name}` — 删除技能

六、前端开发要求

1. 设计理念与布局架构

前端采用 **多页面 IDE 风格** 布局，共 3 个页面：

- **对话页 (/)**: 左侧导航 + 会话列表 | 中间聊天区 + 右侧可选面板 (Raw Messages / Canvas Preview)
- **记忆页 (/memory)**: 左侧导航 | 中间文件卡片列表 | 右侧 Monaco Editor
- **技能页 (/skills)**: 左侧导航 | 中间技能卡片库 | 右侧 Monaco Editor

2. 技术栈

- **框架**: Next.js 14+ (App Router), TypeScript
- **UI**: Tailwind CSS, Lucide Icons, Space Grotesk 字体
- **Editor**: Monaco Editor (支持 Light/Dark 主题自动切换)
- **主题**: 支持 Dark/Light 双主题切换

3. UI/UX 风格规范

- **色调**: Bronze/Gold 双主题设计
 - 浅色主题 (Light): Cream (#fcfaf7) 背景, Bronze (#B8860B) 强调色
 - 暗色主题 (Dark): Charcoal (#121212) 背景, Gold (#D4AF37) 强调色
- **导航栏**: 顶部固定 (h-14)
 - 左侧: 学习模式按钮
 - 中央: "Fufan OpenClaw" Logo

- 右侧：主题切换、系统状态、"赋范空间" (链接至 <https://fufan.ai>)
- 字体: Space Grotesk (via Google Fonts / next/font)

4. 核心功能

- **Canvas (A2UI)**: Agent 可输出 `<openclaw-canvas>` 标签包裹的 HTML，前端在右侧 Canvas 面板中通过 iframe 实时渲染
- **学习模式**: 左侧可展开的学习辅助面板，展示 Agent 操作日志和教学信息
- **Raw Messages 面板**: 右侧滑入式面板，显示完整的 System Prompt、用户消息和 Agent 消息
- **AI 优化**: 记忆文件支持 AI 辅助优化 (对比原文与优化建议)
- **技能商店**: 支持从 GitHub 浏览和安装社区技能
- **对话压缩**: 支持压缩历史对话为摘要
- **RAG 模式**: 支持开启记忆检索增强模式

七、项目目录结构参考

```
fufan-openclaw/
├── backend/                # FastAPI + LangChain/LangGraph
│   ├── app.py             # 入口文件 (Port 8002)
│   ├── config.py          # JSON 配置管理 (RAG 模式等)
│   ├── api/               # API 路由
│   │   ├── chat.py        # SSE 流式对话
│   │   ├── files.py       # 文件读写 + 技能列表
│   │   ├── sessions.py    # 会话 CRUD
│   │   ├── tokens.py      # Token 统计
│   │   ├── compress.py    # 对话压缩
│   │   └── config_api.py  # RAG 模式配置
│   ├── graph/             # LangGraph 状态机定义
│   │   ├── agent.py       # Agent 管理器
│   │   ├── prompt_builder.py # System Prompt 拼接
│   │   ├── session_manager.py # 会话持久化
│   │   └── memory_indexer.py # MEMORY.md 向量索引
│   ├── tools/             # 6 Core Tools
│   │   ├── terminal_tool.py
│   │   ├── python_repl_tool.py
│   │   ├── fetch_url_tool.py
│   │   ├── read_file_tool.py
│   │   ├── search_knowledge_tool.py
│   │   ├── browser_use_tool.py
│   │   └── skills_scanner.py
│   ├── utils/
│   │   └── encoding.py     # Windows GBK 编码安全读取
│   ├── workspace/         # System Prompts
│   │   ├── SOUL.md
│   │   ├── IDENTITY.md
│   │   ├── USER.md
│   │   └── AGENTS.md
│   ├── skills/            # Agent Skills 文件夹
│   ├── memory/            # MEMORY.md + logs/
│   ├── sessions/          # JSON 会话记录 + archive/
│   ├── SKILLS_SNAPSHOT.md # 自动生成的技能快照
│   └── requirements.txt
```



```
|
├─ frontend/                                # Next.js 14+ (App Router)
|   └─ src/
|       └─ app/
|           ├─ layout.tsx                    # 根布局 (ThemeProvider + AppProvider)
|           ├─ page.tsx                      # 对话页面
|           ├─ memory/page.tsx               # 记忆管理页面
|           └─ skills/page.tsx               # 技能管理页面
|       └─ components/
|           ├─ layout/                       # Header, Sidebar, LearnPanel
|           └─ chat/                         # ChatPanel, ChatMessage, ChatInput,
ThoughtChain, RawMessagesPanel, CanvasPanel
|       └─ memory/                           # MemoryFileList, MemoryEditor, OptimizeModal
|       └─ skills/                           # SkillLibrary, SkillEditor, NewSkillModal,
SkillStore
|       └─ shared/                           # ConfirmDialog
|       └─ lib/
|           ├─ api.ts                        # Backend API 客户端
|           ├─ store.tsx                     # React Context 全局状态
|           └─ theme.tsx                     # 主题切换 Context
|       └─ package.json
|
├─ CLAUDE.md                                # Claude Code 指导文件
└─ FuFan-OpenClaw 开发需求文档 (PRD).md
```